



Desarrollo de Aplicaciones con APIs de IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Visión por computador aplicada a proyectos de programación



- I. ¿Qué permite la visión por computador?
- II. Casos de uso principales
- III. Visión clásica vs visión con APIs modernas
- IV. Ejemplo: descripción de imagen (Node.js)
- V. Ejemplo: OCR con Python (Gemini)
- VI. Ejemplo: detección de objetos (Hugging Face)
- VII. 5 claves y ¿Qué técnica usar?
- VIII. Buenas prácticas y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

Idea clave

La visión por computador(CV) permite que una aplicación interprete imágenes igual que el NLP interpreta texto. Gracias a las APIs modernas, cualquier software puede analizar imágenes sin entrenar modelos propios.

- Analizar imágenes
- Detectar objetos
- Extraer texto (OCR)
- Describir contenido
- Clasificar imágenes
- Detectar contenido inapropiado
- Combinar texto + imagen (multimodal)

Idea clave

Detectar objetos

Identifica elementos concretos dentro de una imagen.

Ejemplo: "Hay 2 coches, 1 semáforo y 1 peatón".

Extraer texto (OCR)

Convierte texto dentro de una imagen en texto digital.

Ejemplo: leer un ticket, una matrícula o un cartel.

Describir contenido

Genera una descripción en lenguaje natural de lo que aparece en la imagen.

Ejemplo: "Una persona usando un portátil en una mesa de madera".

Idea clave

Clasificar imágenes

Asigna categorías según el contenido visual.

Ejemplo: "gato", "perro", "paisaje", "producto", "documento".

Detectar contenido inapropiado

Filtra imágenes con violencia, desnudos, contenido sensible o peligroso.

Ejemplo: moderación automática en plataformas con subida de imágenes.

Combinar texto + imagen (multimodal)

El modelo entiende la imagen y el texto juntos.

Ejemplo: "¿Qué dice este cartel y qué crees que significa en este contexto?"

■ ¿Por qué es útil para proyectos de programación?

Porque permite automatizar tareas que antes requerían intervención humana:

- Leer documentos físicos
 - Analizar fotos enviadas por usuarios
 - Automatizar inventarios con imágenes
 - Validar capturas de pantalla
 - Extraer datos de facturas o tickets
 - Crear asistentes que entienden imágenes
 - Construir sistemas multimodales (texto + imagen)
- Todo sin entrenar modelos, sin GPUs y sin datasets propios.

Visión por computador aplicada a proyectos de programación



■ Flujo típico de visión por computador en una app

El usuario sube una imagen → Foto, captura, documento, producto, etc.

La aplicación envía la imagen a la API → Igual que con NLP, pero enviando un archivo o una URL.

La API analiza la imagen → Detecta objetos, texto, categorías o genera descripciones.

Devuelve una respuesta estructurada → JSON con objetos, coordenadas, texto extraído o etiquetas.

La aplicación actúa según el resultado → Mostrar información, validar datos, activar acciones, etc.

■ Ejemplo sencillo

Entrada: una foto de un ticket

Petición a la API: "Extrae el total y la fecha del ticket"

Salida JSON:

```
{  
  "fecha": "2024-03-12",  
  "total": "23.50"  
}
```

Acción en la app: → Rellenar automáticamente un formulario de gastos.



Visión por computador aplicada a proyectos de programación



Analogía

Nos imaginamos que nuestra aplicación necesita:

- Reconocer objetos en fotos
- Leer texto de imágenes (OCR)
- Detectar contenido inapropiado
- Clasificar imágenes por categorías
- Describir lo que aparece en una escena
- Combinar texto + imagen para responder preguntas

■ Si lo hicieras entrenando tu propio sistema de visión...

Sería como montar un equipo enorme desde cero, formado por:

- **Fotógrafos** Para entender iluminación, encuadre y calidad de imagen.
- **Analistas de imagen** Para identificar objetos, patrones y características visuales.
- **Expertos en OCR** Para leer texto en fotos, documentos, tickets o carteles.
- **Especialistas en seguridad y moderación** Para detectar contenido sensible, inapropiado o peligroso.
- **Ingenieros de IA** Para entrenar modelos, ajustar parámetros, limpiar datasets y mantener servidores con GPUs.



Visión por computador aplicada a proyectos de programación



Tendrías que:

- Reunir miles de imágenes etiquetadas
- Entrenar modelos pesados
- Ajustarlos continuamente
- Mantener infraestructura costosa
- Gestionar errores, sesgos y actualizaciones

Es decir: mucho tiempo, mucho dinero y mucha complejidad técnica.

■ Con APIs modernas de visión por computador...

Es como contratar a ese equipo ya formado, disponible:

- 24/7
- Escalable a miles de imágenes
- Sin necesidad de entrenamiento
- Sin infraestructura propia
- Con mejoras automáticas del proveedor
- Con resultados inmediatos mediante una simple llamada a la API

Tu aplicación solo necesita enviar una imagen y recibe:

- Objetos detectados
- Texto extraído
- Categorías
- Descripciones
- Alertas de contenido sensible
- Respuestas multimodales (texto + imagen)

Todo sin entrenar modelos propios.



3.1— ¿Qué permite la visión por computador?

Capacidad CV	¿Qué hace?	Ejemplo práctico en software
Clasificar imágenes	Asigna categorías según el contenido visual.	App que etiqueta fotos como "producto", "documento", "persona".
Detectar objetos	Encuentra elementos concretos en la imagen.	App que cuenta coches en una calle o detecta herramientas en una mesa.
Extraer texto (OCR)	Convierte texto en imagen a texto digital.	App que lee tickets, facturas, matrículas o carteles.
Describir imágenes	Genera una descripción en lenguaje natural.	App que resume lo que aparece en una foto para personas con baja visión.
Analizar contenido inapropiado	Detecta violencia, desnudos o contenido sensible.	Moderación automática en redes sociales o plataformas educativas.
Identificar rostros (según proveedor)	Reconoce personas en imágenes.	App que etiqueta fotos o valida identidad en accesos.
Leer documentos escaneados	Interpreta formularios, PDFs o imágenes de texto.	App que extrae datos de contratos, recibos o fichas médicas.
Entender imágenes + texto (multimodal)	Combina lo visual con lo textual para responder o razonar.	App que responde: "¿Qué dice este cartel y qué significa?"

■ 3.1— ¿Qué permite la visión por computador?

■ ¿Cómo se aplican las capacidades de visión por computador en proyectos reales?

No es solo ver imágenes, es entenderlas.

- Cada capacidad puede activarse con una simple llamada a una API.
- No necesitas entrenar modelos ni tener datasets propios.
- Puedes construir apps que “ven” y “razonan” con imágenes igual que con texto.

■ Ejemplo de flujo multimodal

Entrada: imagen de un cartel + pregunta: “¿Qué dice y qué significa?”

API: analiza imagen + interpreta texto + responde en lenguaje natural

Salida:

“El cartel dice ‘Prohibido el paso’. Es una advertencia de acceso restringido.”



3.1— ¿Qué permite la visión por computador?



LA VISIÓN POR COMPUTADOR ES MÁS QUE "VER IMÁGENES"

Cuando hablamos de visión por computador en el contexto de desarrollo de software, no nos referimos solo a que la aplicación "vea" una imagen. Nos referimos a capacidades que antes requerían equipos enteros de expertos en visión artificial, fotógrafos y anotadores de datos.

ANALIZAR IMÁGENES: DETECTAR, CLASIFICAR Y COMPRENDER

El análisis de imágenes no es simplemente "ver qué hay". Incluye múltiples capas:

- Análisis de contenido: ¿Qué objetos aparecen? ¿En qué cantidad?
- Análisis de contexto: ¿Dónde ocurre la escena? ¿Interior o exterior?
- Análisis de calidad: ¿La imagen está borrosa? ¿Tiene buena iluminación?
- Análisis de texto: ¿Hay texto incrustado en la imagen?

Por ejemplo, una foto de una cocina puede contener: objetos (nevera, horno, mesa), contexto (interior, doméstico), calidad (iluminación natural), y texto (etiquetas de productos, pantalla del microondas).



3.1— ¿Qué permite la visión por computador?



DETECCIÓN DE OBJETOS: LOCALIZAR, NO SOLO IDENTIFICAR

La detección de objetos va más allá de decir "hay un coche". Un buen sistema debe:

- Indicar cuántos objetos de cada tipo hay
- Proporcionar coordenadas (bounding boxes) para localizarlos
- Dar una puntuación de confianza para cada detección
- Distinguir entre objetos superpuestos o parcialmente ocultos

EXTRACCIÓN DE TEXTO (OCR): LEER EN CUALQUIER FORMATO

El OCR moderno no es simplemente "extraer letras". Puede:

- Leer texto en cualquier orientación (rotado, inclinado)
- Reconocer diferentes fuentes y tamaños
- Mantener la estructura (tablas,



3.1— ¿Qué permite la visión por computador?



DESCRIPCIÓN DE IMÁGENES: EXPLICAR EN LENGUAJE NATURAL

La descripción automática no es solo "una persona en una playa". Un buen sistema puede:

- Describir acciones ("una persona corriendo", "dos personas conversando")
- Mencionar relaciones espaciales ("el perro está a la izquierda del niño")
- Adaptar el nivel de detalle (breve vs detallado)
- Identificar emociones o estados ("persona feliz", "entorno tranquilo")

MODERACIÓN DE CONTENIDO: PROTEGER AL USUARIO

La moderación visual puede detectar automáticamente:

- Contenido violento: armas, sangre, peleas
- Contenido adulto: desnudos, actos sexuales
- Contenido peligroso: drogas, armas, autolesiones
- Contenido ofensivo: símbolos de odio, gestos ofensivos



3.1— ¿Qué permite la visión por computador?



DESCRIPCIÓN DE IMÁGENES: EXPLICAR EN LENGUAJE NATURAL

La descripción automática no es solo "una persona en una playa". Un buen sistema puede:

- Describir acciones ("una persona corriendo", "dos personas conversando")
- Mencionar relaciones espaciales ("el perro está a la izquierda del niño")
- Adaptar el nivel de detalle (breve vs detallado)
- Identificar emociones o estados ("persona feliz", "entorno tranquilo")

MODERACIÓN DE CONTENIDO: PROTEGER AL USUARIO

La moderación visual puede detectar automáticamente:

- Contenido violento: armas, sangre, peleas
- Contenido adulto: desnudos, actos sexuales
- Contenido peligroso: drogas, armas, autolesiones
- Contenido ofensivo: símbolos de odio, gestos ofensivos

3.2— Casos de uso principales

Tipo de tarea visual	Objetivo funcional	Técnica CV	API recomendada según enfoque
Digitalizar texto visual	Convertir texto en imagen a texto digital	OCR	Gemini (multimodal), Azure Vision (precisión en documentos)
Etiquetar contenido visual	Asignar categoría o tipo a una imagen	Clasificación	Hugging Face ViT (modelo abierto), OpenAI (zero-shot)
Localizar elementos específicos	Detectar y posicionar objetos en la imagen	Detección de objetos	YOLO (modelo clásico), Azure , HF
Generar descripciones automáticas	Explicar lo que aparece en una imagen	Descripción	GPT-4o (multimodal), Claude 3 (contexto largo)
Filtrar contenido sensible	Detectar violencia, desnudos o contenido inapropiado	Moderación visual	OpenAI Moderation , Azure Content Safety
Interpretar imagen + texto	Razonar sobre lo visual y lo textual juntos	Multimodal	GPT-4o , Gemini , Claude 3

■ 3.2—Casos de uso principales

■ ¿Qué técnica usar según el tipo de tarea visual?

Cada técnica responde a una **necesidad funcional concreta** en el software. El tipo de API depende del nivel de control, coste, precisión y contexto del **proyecto**.

El programador debe aprender a **elegir la técnica adecuada según el objetivo**, no solo por nombre o moda.

■ Ejemplo de aplicación combinada

Caso: App que procesa facturas escaneadas

- Usa **OCR** para extraer texto
- Usa **clasificación** para detectar si es factura, ticket o recibo
- Usa **moderación** si el documento contiene imágenes sensibles

■ 3.2—Casos de uso principales

■ ¿Qué técnica usar según el tipo de tarea visual?

Cada técnica responde a una **necesidad funcional concreta** en el software. El tipo de API depende del nivel de control, coste, precisión y contexto del **proyecto**.

El programador debe aprender a **elegir la técnica adecuada según el objetivo**, no solo por nombre o moda.

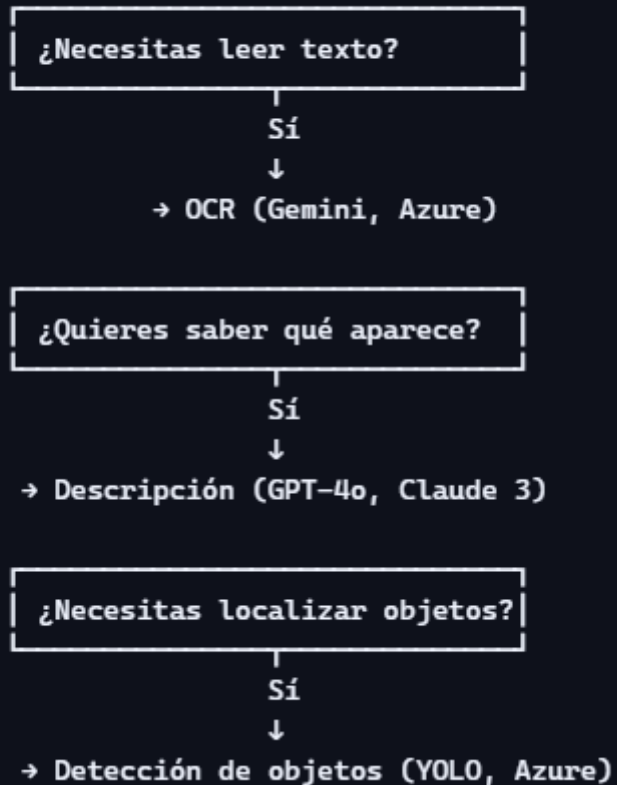
■ Ejemplo de aplicación combinada

Caso: App que procesa facturas escaneadas

- Usa **OCR** para extraer texto
- Usa **clasificación** para detectar si es factura, ticket o recibo
- Usa **moderación** si el documento contiene imágenes sensibles

Visión por computador aplicada a proyectos de programación

¿QUÉ QUIERES HACER CON UNA IMAGEN?



Si tu app necesita LEER → usa OCR
Si necesita ENTENDER QUÉ HAY → usa Clasificación
Si necesita LOCALIZAR → usa Detección de objetos
Si necesita EXPLICAR → usa Descripción
Si necesita PROTEGER → usa Moderación
Si necesita RAZONAR → usa Multimodal

■ 3.2—Casos de uso principales

1. AUTOMATIZACIÓN DE FACTURACIÓN Y GASTOS

Una aplicación de gestión de gastos donde los usuarios:

- Fotografían tickets de compra
- El sistema extrae automáticamente: fecha, total, impuestos, comercio
- Rellena el formulario de gastos automáticamente
- Archiva la imagen como justificante

Ejemplo real: App de gastos de empresa donde 10.000 empleados suben tickets. El procesamiento manual costaría miles de horas. Con visión por computador, es automático y en segundos. ~~tario~~ manual.

■ 3.2—Casos de uso principales

2. CONTROL DE INVENTARIO CON FOTOS

Un sistema de inventario para almacenes que:

- Analiza fotos de estanterías
- Detecta productos agotados o mal colocados
- Cuenta unidades automáticamente
- Genera alertas de reposición

Ejemplo real: Una cadena de supermercados usa fotos de pasillos para detectar productos faltantes sin necesidad de inventario manual.1.

■ 3.2—Casos de uso principales

3. VALIDACIÓN DE DOCUMENTOS

Un sistema de onboarding que:

- Verifica que el DNI subido es legible
- Extrae nombre, número y fecha de caducidad
- Detecta si la foto es una foto de una pantalla (posible fraude)
- Compara con selfie para verificar identidad

4. ACCESIBILIDAD PARA PERSONAS CON DISCAPACIDAD VISUAL

Una aplicación móvil que:

- Describe el entorno: "hay una silla a tu izquierda"
- Lee texto en carteles, menús o documentos
- Identifica obstáculos o peligros
- Reconoce caras de amigos o familiares

■ 3.2—Casos de uso principales

5. MODERACIÓN AUTOMÁTICA DE CONTENIDO

Una plataforma de redes sociales que:

- Analiza todas las imágenes subidas
- Bloquea contenido violento o adulto
- Detecta spam visual (logos, códigos QR)
- Alerta sobre contenido potencialmente peligroso

6. CONTROL DE CALIDAD INDUSTRIAL

Un sistema de fabricación que:

- Analiza fotos de productos en línea de producción
- Detecta defectos, arañazos, piezas mal montadas
- Rechaza automáticamente productos defectuosos
- Genera estadísticas de calidad

3.3—Visión clásica vs visión con APIs modernas



Enfoque	Cuándo usarlo	Qué exige al desarrollador	Qué aporta al proyecto
Visión clásica (OpenCV, modelos entrenados)	Cuando necesitas control fino, trabajar offline o cumplir requisitos estrictos (industria, seguridad, hardware dedicado).	- Preparar datasets	

- Entrenar modelos
- Ajustar parámetros
- Mantener infraestructura
- Conocer visión tradicional | - Control total del pipeline
- Funcionamiento sin conexión
- Personalización profunda | |
- **APIs modernas** (GPT-4o, Gemini, Claude, Azure Vision) |
- Cuando necesitas resultados rápidos, multimodalidad o integrar visión sin experiencia en IA. | - Diseñar prompts
- Enviar imágenes a la API
- Validar respuestas
- Integrar JSON en la app | - Velocidad de desarrollo
- Resultados avanzados sin entrenar modelos
- Capacidades multimodales (imagen + texto) |

■ 3.3—Visión clásica vs visión con APIs modernas



■ Idea clave

- La visión clásica es ingeniería.
- La visión con APIs modernas es integración.

■ Ejemplos que ayudan a entender la diferencia

✓ Visión clásica

Un sistema industrial que debe detectar defectos en piezas metálicas con precisión milimétrica. → Necesita modelos entrenados con imágenes reales de la fábrica.

✓ APIs modernas

Una app que permite subir una foto de un ticket y extraer el total. → Basta con una llamada a la API, sin entrenar nada.

3.3—Visión clásica vs visión con APIs modernas

CUÁNDO USAR VISIÓN CLÁSICA:

- Necesitas funcionamiento sin conexión a internet
- Tienes requisitos de latencia muy estrictos (milisegundos)
- Procesas volúmenes masivos (millones de imágenes por hora)
- Necesitas privacidad total (datos médicos, militares)
- Tienes un caso de uso muy específico no cubierto por APIs generales
- Requieres control total sobre el pipeline de procesamiento

EJEMPLO: Un sistema de inspección industrial que debe detectar defectos en piezas metálicas con precisión milimétrica. No hay API genérica para "defectos en tu pieza específica".

■ 3.3—Visión clásica vs visión con APIs modernas

DESVENTAJAS DE VISIÓN CLÁSICA:

- Necesitas miles (o millones) de imágenes etiquetadas
- El etiquetado manual es caro y lento
- Necesitas GPUs potentes y mantenimiento de infraestructura
- Los modelos requieren actualizaciones periódicas
- El tiempo de desarrollo es de meses, no de días

APIS MODERNAS: RÁPIDAS, ECONÓMICAS Y SIN MANTENIMIENTO

■ 3.3—Visión clásica vs visión con APIs modernas

VENTAJAS:

- Sin necesidad de datasets ni entrenamiento
- Resultados inmediatos (segundos)
- Escalabilidad automática
- Mejoras continuas sin acción por tu parte
- Capacidades multimodales (imagen + texto)

CUÁNDO USAR APIS MODERNAS:

- Prototipos y pruebas de concepto
- Aplicaciones con volumen moderado (miles de imágenes/día)
- Casos de uso generales (documentos, objetos comunes, moderación)
- Necesitas respuestas en lenguaje natural sobre imágenes
- El tiempo de desarrollo es crítico

3.3—Visión clásica vs visión con APIs modernas



Aspecto	Visión Clásica	APIs Modernas
Tiempo de desarrollo	3-6 meses	1-3 días
Coste inicial	Alto (GPUs, etiquetado)	Bajo (pago por uso)
Precisión en casos generales	Variable	Excelente
Precisión en casos específicos	Puede ser superior	Limitada
Mantenimiento	Continuo	Casi nulo
Funcionamiento offline	Sí	No
Coste por imagen	Bajo a gran escala	Pequeño pero constante

IDEA CLAVE: No es "uno contra otro". La mejor estrategia suele ser híbrida:

- APIs para tareas generales (clasificar tipo de documento)
- Modelos clásicos para tareas específicas (detectar defectos en tu pieza)

3.4—Ejemplo: descripción de imagen (Node.js)



```
const response = await client.chat.completions.create({  
  model: "gpt-4o-mini",  
  messages: [  
    {  
      role: "user",  
      content: [  
        { type: "input_text", text: "Describe esta imagen" },  
        { type: "input_image", image_url: "https://..." }  
      ]  
    }  
  ]  
});
```

```
const response = await client.chat.completions.create({
```

Qué hace: Lanza una petición a la API de chat (multimodal) de OpenAI.
Clave : Es una llamada asíncrona (await), por lo que esto va dentro de una función async.
Idea: "Voy a pedirle al modelo que describa una imagen que le voy a enviar".



3.4—Ejemplo: descripción de imagen (Node.js)



```
const response = await client.chat.completions.create({  
  model: "gpt-4o-mini",  
  messages: [  
    {  
      role: "user",  
      content: [  
        { type: "input_text", text: "Describe esta imagen" },  
        { type: "input_image", image_url: "https://..." }  
      ]  
    }  
  ]  
});
```

```
const response = await client.chat.completions.create({
```

Qué hace: Lanza una petición a la API de chat (multimodal) de OpenAI.
Clave : Es una llamada asíncrona (await), por lo que esto va dentro de una función async.
Idea: "Voy a pedirle al modelo que describa una imagen que le voy a enviar".



3.4—Ejemplo: descripción de imagen (Node.js)

```
model: "gpt-4o-mini",
```



Qué hace: Indica qué modelo usar.

Por qué este: gpt-4o-mini es más barato y rápido, suficiente para tareas de descripción de imagen.

Mensaje "No siempre hace falta el modelo más grande para tareas concretas".

```
messages: [
  {
    role: "user",
    content: [
      { type: "input_text", text: "Describe esta imagen" },
      { type: "input_image", image_url: "https://..." }
    ]
  }
]
```

Aquí está la parte interesante: **mensaje multimodal**.

• **role: "user"** Indica que este mensaje viene del usuario (igual que en chat normal).

• **content: [...]** En lugar de ser solo texto, es un array con **bloques de contenido**.

a) Bloque de texto

• **Qué hace:** Le dice al modelo qué queremos que haga con la imagen.

• **Idea clave:** El prompt sigue siendo importante, aunque haya imagen.

b) Bloque de imagen

• **Qué hace:** Le pasa al modelo la URL de la imagen que queremos que describa.

• **Para los programadores:**

- Puede ser una URL pública.

- En otros casos, se puede enviar la imagen como base64 (según SDK).



3.4—Ejemplo: descripción de imagen (Node.js)

Respuesta del modelo

```
console.log(response.choices[0].message.content);
```

Qué obtienen: Un texto con la descripción de la imagen.

Ejemplo esperado: "Se ve una persona sentada frente a un portátil en una mesa de madera, con una taza de café al lado."

- Están haciendo lo mismo que en NLP, pero ahora el

`content` puede incluir **texto + imagen**.

- La lógica de integración es la misma:

1. Preparar el prompt

2. Enviar la petición

3. Recibir la respuesta

4. Usarla en la app (mostrar, guardar, procesar...)

3.5—Ejemplo: OCR con Python (Gemini)



```
response = model.generate_content([
    "Extrae el texto de esta imagen:",
    Image.open("documento.png")
])
print(response.text)
```

```
response = model.generate_content([
```

Qué hace: Llama al modelo multimodal para generar contenido a partir de texto + imagen.

Idea clave para programadores:

generate_content es la función que permite enviar múltiples tipos de entrada. El modelo entiende que va a recibir instrucciones y una imagen.



3.5—Ejemplo: OCR con Python (Gemini)



```
response = model.generate_content([  
    "Extrae el texto de esta imagen:",  
    Image.open("documento.png")  
)  
print(response.text)
```

Construcción del prompt multimodal

"Extrae el texto de esta imagen:",

Qué hace: Es la instrucción textual.

Importante: Aunque haya imagen, el prompt sigue siendo fundamental.

Mensaje "La IA no adivina lo que quieres. Hay que decirle exactamente qué hacer."



3.5—Ejemplo: OCR con Python (Gemini)



```
response = model.generate_content([
    "Extrae el texto de esta imagen:",
    Image.open("documento.png")
])
print(response.text)
```

Carga de la imagen

`Image.open("documento.png")`

Qué hace: Abre el archivo de imagen desde el disco.

Por qué es relevante:

Gemini acepta imágenes directamente como objetos PIL.

Esto evita tener que convertir a base64 o URLs.

La imagen se envía como parte del input multimodal.

El modelo la analiza igual que un humano leería un documento.

3.5—Ejemplo: OCR con Python (Gemini)



```
response = model.generate_content([
    "Extrae el texto de esta imagen:",
    Image.open("documento.png")
])
print(response.text)
```

Lectura del resultado

```
print(response.text)
```

Qué hace: Imprime el texto extraído por el modelo.

Qué esperar: El contenido OCR limpio.

Puede incluir saltos de línea, números, direcciones, etc.

Ejemplo típico:

```
CódigoFACTURA Nº 2341
Total: 23,50 €
Fecha: 12/03/2024.
```



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

■ 3.5—Ejemplo: OCR con Python (Gemini)

- Es OCR sin entrenar modelos: no hace falta Tesseract,  dicampus datasets ni preprocesado.
- El flujo es igual que en NLP, solo cambia el tipo de entrada.
- El modelo interpreta la imagen como un documento, no como una foto cualquiera.
- La salida es texto puro, listo para procesar, almacenar o validar.
- La clave está en el prompt: "Extrae el texto", "Devuelve solo el contenido", "Formatea en JSON", etc.



3.6—Ejemplo: detección de objetos (Hugging Face)



```
from transformers import pipeline
detector = pipeline("object-detection")
result = detector("imagen.jpg")
print(result)
```

Importación del módulo

`from transformers import pipeline`

- Qué hace: Importa la función pipeline del ecosistema Hugging Face Transformers.
- Por qué es importante:
 - pipeline es la forma más sencilla de usar modelos preentrenados sin configurar nada.
 - Evita tener que cargar modelos manualmente, tokenizadores, pesos, etc.

Mensaje

"Hugging Face te da un atajo: una línea para cargar un modelo completo."

3.6—Ejemplo: detección de objetos (Hugging Face)



```
from transformers import pipeline
detector = pipeline("object-detection")
result = detector("imagen.jpg")
print(result)
```

Creación del pipeline de detección de objetos

detector = pipeline("object-detection")

Qué hace: Crea un pipeline especializado en detección de objetos.

Qué significa "object-detection":

- El modelo no solo identifica qué objetos hay, sino dónde están (coordenadas).

Qué modelo usa por defecto:

- Normalmente un modelo tipo DETR o YOLO preentrenado, según la versión instalada.

Idea clave

Con una sola línea tienes un modelo de visión por computador listo para usar."



3.6—Ejemplo: detección de objetos (Hugging Face)



```
from transformers import pipeline
detector = pipeline("object-detection")
result = detector("imagen.jpg")
print(result)
```

Ejecutar la detección sobre una imagen

`result = detector("imagen.jpg")`

Carga la imagen desde el archivo "imagen.jpg".

La envía al modelo para analizarla. El modelo devuelve una lista de objetos detectados con:

- label → nombre del objeto
- score → probabilidad/confianza
- box → coordenadas del objeto (xmin, ymin, xmax, ymax)

3.6—Ejemplo: detección de objetos (Hugging Face)



```
[
  {
    'score': 0.98,
    'label': 'person',
    'box': {'xmin': 34, 'ymin': 12, 'xmax': 180, 'ymax': 300}
  },
  {
    'score': 0.87,
    'label': 'dog',
    'box': {'xmin': 200, 'ymin': 100, 'xmax': 350, 'ymax': 280}
  }
]
```

Ejemplo típico de salida:

3.6—Ejemplo: detección de objetos (Hugging Face)



Mostrar el resultado

`print(result)`

- Imprime la lista de objetos detectados.
- Esta salida es **JSON-like**, por lo que se puede:
 - dibujar cajas en la imagen,
 - filtrar objetos,
 - contar elementos,
 - integrarlo en un backend o dashboard.
- Hugging Face permite usar modelos de visión **sin entrenar nada**.
- La detección de objetos devuelve **información estructurada** lista para usar.
- Es ideal para:
 - inventarios automáticos,
 - análisis de imágenes,
 - seguridad,
 - proyectos educativos,
 - prototipos rápidos.



3.6—Ejemplo: descripción de imagen con Python



DESCRIPCIÓN DE IMÁGENES: EXPLICANDO LO QUE LA IA "VE"

La descripción de imágenes es una de las capacidades más impresionantes de los modelos multimodales. Pueden describir escenas completas, identificar objetos, acciones, emociones y relaciones espaciales.

QUÉ PUEDES PEDIR EN UNA DESCRIPCIÓN:

- Descripción general: "Describe esta imagen"
- Descripción detallada: "Describe cada elemento en detalle"
- Descripción para ciegos: "Describe como si el usuario no pudiera ver"
- Descripción técnica: "Describe en términos fotográficos"
- Descripción creativa: "Describe como si fuera un poema"
- Descripción funcional: "Describe qué acciones puede realizar un robot aquí"



3.6—Ejemplo: descripción de imagen con Python

DESCRIPCIÓN DE IMÁGENES: EXPLICANDO LO QUE LA IA "VE"



FORMATOS DE SALIDA ÚTILES:

Puedes pedir la descripción en diferentes formatos:

- Texto libre: para mostrar al usuario
- JSON estructurado: para procesamiento automático
- Lista de etiquetas: para búsqueda y filtrado
- Con coordenadas: "hay un perro en la esquina superior izquierda"

APLICACIONES PRÁCTICAS:

- Accesibilidad: describir imágenes para personas con discapacidad visual
- SEO: generar automáticamente texto alternativo para imágenes de sitios web
- Organización: clasificar y describir bibliotecas de fotos
- Redes sociales: generar sugerencias de texto para publicaciones
- Educación: crear descripciones para material didáctico

■ 3.7—5 claves y ¿Qué técnica usar?



1. Qué aporta la visión por computador hoy

Permite que las aplicaciones interpreten imágenes igual que el NLP interpreta texto.

Gracias a APIs modernas, podemos analizar imágenes sin entrenar modelos propios.

Soporta tareas como OCR, clasificación, detección, descripción, moderación y análisis multimodal.

2. Cómo elegir la técnica adecuada

- ☐ **OCR** → cuando el proyecto necesita leer documentos o texto en imágenes.
- ☐ **Clasificación** → cuando hay que organizar o etiquetar imágenes automáticamente.
- ☐ **Detección de objetos** → cuando necesitamos localizar elementos concretos.
- ☐ **Descripción de imágenes** → cuando la app debe explicar lo que ve.
- ☐ **Moderación visual** → cuando hay que filtrar contenido sensible.
- ☐ **Multimodal** → cuando la app debe razonar con imagen + texto.



■ 3.7—5 claves y ¿Qué técnica usar?



3. Visión clásica vs APIs modernas

- **Visión clásica** → control total, personalización, uso offline; requiere datasets y mantenimiento.
- **APIs modernas** → rápidas, multimodales y sin entrenamiento; ideales para prototipos y apps reales.

4. Cómo se integra en código

Node.js: enviar imagen + prompt → recibir descripción.

Python (Gemini): cargar imagen → extraer texto (OCR).

Hugging Face: pipeline directo → detección de objetos.

La lógica siempre es la misma: **prompt** → **imagen** → **respuesta estructurada**.

5. Idea clave para programadores

No necesitas ser experto en visión por computador para construir apps que “ven”. Solo necesitas saber qué técnica elegir y cómo integrarla.



■ 3.8—Buenas prácticas y errores comunes



Optimizar imágenes antes de enviarlas

- Reducir resolución y peso mejora velocidad y reduce coste.
- Las APIs no necesitan fotos de 8-12 MB para entender el contenido.

Solicitar salidas estructuradas (JSON)

- Facilita validar coordenadas, etiquetas y texto extraído.
- Evita respuestas ambiguas o difíciles de procesar.

Validar siempre la respuesta del modelo

- Comprobar que las coordenadas existen, que el OCR tiene sentido y que las etiquetas son coherentes.
- La visión puede fallar con sombras, reflejos o baja calidad.



■ 3.8—Buenas prácticas y errores comunes



Elegir el modelo adecuado según la tarea

- Modelos pequeños → clasificación simple.
- Modelos multimodales → razonamiento complejo.
- No usar "martillos grandes" para clavos pequeños.

Evitar enviar imágenes sensibles

- Privacidad, cumplimiento y ética: filtrar antes de enviar.
- Especialmente importante en educación, salud y RRHH.

Controlar costes

- Las imágenes consumen más tokens que el texto.
- Reducir tamaño, pedir respuestas cortas y limitar llamadas.





3.8—y errores comunes



Enviar imágenes demasiado grandes

Aumenta coste, ralentiza la respuesta y no mejora la precisión.

No especificar el formato de salida

El modelo puede devolver texto libre, difícil de procesar.

Siempre pedir JSON o un formato claro.

Confiar ciegamente en el OCR

El OCR puede confundir números, fechas o saltos de línea.

Validar siempre antes de usar en la app.





3.8—y errores comunes



No revisar las coordenadas de detección

Algunas detecciones pueden estar fuera de rango o solapadas.
Importante en apps que dibujan bounding boxes.

Usar modelos multimodales para tareas triviales

Clasificar "gato/perro" no requiere GPT-4o.
Aumenta coste sin aportar valor.

No controlar contenido inapropiado

Si los usuarios suben imágenes, siempre moderar antes de procesar.
Evita problemas legales y éticos.





Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

